САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа 2

Выполнила:

Лорс Алина, 474321

Группа J3214

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Цель работы:

Связать приложение ЛР1 с внешними источниками данных и перенести операции CRUD в mock API с авторизацией.

-Внешнее API,которое использовалось Frankfurter API для получения курсов валют.

-Закрытые ресурсы Коллекции accounts, transactions, budgets, integrations, importRules, imports, users.

-Основные технологииHTML, CSS, Bootstrap, JavaScript, fetch, JSON Server, json-server-auth, localStorage

1. Задача:

ЛР1 привязать его к внешнему API средствами fetch, axios или XHR(Согласно заданию), реализовать mock API на базе JSON Server, а также подключить авторизацию по аналогии с примерами работы с закрытым API.
У меня: В качестве открытого внешнего API подключён сервис курсов валют Frankfurter. (Он используется для загрузки курсов относительно валюты пользователя). Для внутренней части mock backend на JSON Server,а доступ к пользовательским коллекциям защищён через json-server-auth. выполняются HTTP-запросы через fetch;данные сущностей больше не живут только в интерфейсе, а сохраняются в mock API;регистрация и вход выполняются через /register и /login;после входа закрытые коллекции читаются и изменяются с токеном Bearer;дополнительно используется внешнее API для реальных курсов валют.

2. Общее решение(архитектура)

1. Пользовательское действие в форме или на странице	2. JS-модуль страницы вызывает TarelkaStorage	3. TarelkaStorage делает запрос через fetch	4. JSON Server или Frankfurter API возвращает данные

1 уровень архитектуры лабы- пользовательский интерфейс в каталоге public, содержащий HTML-страницы и клиентские скрипты.

2 Второй уровень - модуль работы с API, который инкапсулирует сетевые запросы, обработку ошибок, авторизацию и синхронизацию локального кэша.

3 уровень - серверная имитация на основе JSON Server, в которой хранится файл базы данных db.json и правила доступа routes.json.

2.1. Структура файлов

package.json — Сценарии запуска и зависимости. Подключены json-server и json-server-auth; сервер стартует на localhost:3000.

db.json — Пустая mock-база. Содержит коллекции users, accounts, transactions, budgets, integrations, importRules, imports.

routes.json — Матрица доступа. Для коллекций задан код 600, то есть доступ только авторизованному владельцу.

public/js/api.js — Низкоуровневые HTTP-запросы. Обёртки над fetch, общий BASE_URL и запросы к внешнему API.

public/js/storage.js — Основной слой данных. Авторизация, синхронизация, CRUD, импорт CSV, отчёты, кэш.

public/js/auth.js — Страницы входа и регистрации. Валидация форм и вызов login/register.

public/js/dashboard.js — Личный кабинет. Карточки метрик, операции, бюджеты, переводы, загрузка курсов.

public/js/transactions.js — Список транзакций. Фильтрация, пагинация, редактирование и удаление.

public/js/integrations.js — Интеграции и импорт. Подключение сервисов, правила импорта, CSV-загрузка.

public/js/reports.js — Визуальные отчёты. Построение графиков на основе данных из mock API.

3. Реализация mock API и авторизации

С точки зрения задания ключевая часть лабы- уход от чисто локальной модели к запросам по HTTP(сайт начал общаться с сервером через запросы)

Для этого в package.json настроен запуск JSON Server с промежуточным модулем json-server-auth. в файле package.json записана команда, которая запускает учебный сервер и подключает к нему вход по логину и паролю.

Сервер одновременно обслуживает и статические файлы интерфейса папка public), и REST-подобные endpoints для данных(адрес для запроса /accounts /transactions /login /register

```
"start": "json-server db.json -m ./node_modules/json-server-auth -r routes.json -s ./public --port 3000"
```

Файл db.json заранее содержит структуру базы данных. При 1запуске все коллекции пустые, но после регистрации нового пользователя приложение автоматически создаёт стартовые записи через API

Файл routes.json описывает закрытые коллекции. Код 600 в json-server-auth ==ресурс доступен только авторизованному пользователю. Таким образом, нельзя получить пользовательские счета, бюджеты или транзакции без успешного входа и JWT-токена.

Коллекция		
users	600	учётные записи и профили
accounts	600	счета пользователя
transactions	600	доходы, расходы и переводы

budgets	600	лимиты по категориям
integrations	600	подключения сервисов
importRules	600	правила обработки импорта
imports	600	история загрузок CSV

3.1. Регистрация и вход пользователя

В auth.js реализованы две отдельные точки входа в систему: initLogin и initRegister. Обе формы проходят клиентскую валидацию: проверяется корректность e-mail, длина пароля, совпадение паролей при регистрации и обязательность согласия с условиями. После успешной проверки данные передаются в модуль TarelkaStorage.При регистрации вызывается метод registerUser это команда, которая создаёт новый аккаунт.

Он отправляет POST-запрос на /register, получает в ответ объект пользователя и accessToken(код подтверждает, что пользователь вошёл.

сохраняет токен в localStorage, создаёт демо-данные через API и затем синхронизирует все коллекции. При входе используется POST-запрос на /login, после чего данные пользователя и токен также попадают в локальную сессию.

```
POST /register -> accessToken + user -> seedUserData() ->
syncCollections()
POST /login    -> accessToken + user -> syncCollections()
```

пароль не сохраняется в currentUser. В storage.js используется функция safeUser, которая убирает поле password из клиентского состояния.

4. Модуль работы с API

4.1. api.js

Файл api.js создаёт глобальный объект window.TarelkaApi. (в браузере создаётся общий объект с командами для работы с API.)

Внутри адрес mock API и адрес внешнего сервиса курсов валют.

Внутри файла реализованы вспомогательные функции parseResponse, extractError, request и requestExternal ==унифицировать обработку ответов и ошибок для разных источников данных.

```
const BASE_URL =
window.location.protocol.startsWith('http') ?
window.location.origin : 'http://localhost:3000';
const EXCHANGE_BASE_URL = 'https://api.frankfurter.dev/v2';
```

Особенность BASE_URL в том, что он зависит от текущего адреса страницы. адрес запросов зависит от того, как именно ты открыла сайт.Поэтому приложение нужно запускать именно как сервер на localhost:3000.

Функция fetchExchangeRates формирует URLSearchParams способ добавить параметры в адрес запроса, фильтрует повторяющиеся валюты и делает запрос к открытому сервису Frankfurter(условие задания о подключении внешнего API реализовано непосредственно через fetch done галочка)

4.2. storage.js

Файл storage.js является мегаважным файлом всей лабы. Он связывает интерфейс, mock API и localStorage. Здесь определены ключи кэша, закрытые коллекции, конечные точки endpoints, обработка токена, формирование заголовков Authorization, синхронизация данных и бизнес-функции для всех разделов приложения.

- ✓ что хранить в браузере
- ✓ какие данные защищены
- ✓ по каким адресам отправлять запросы
- ✓ как работать с кодом доступа после входа
- ✓ как загружать и обновлять данные
- ✓ как выполнять действия пользователя

- authHeaders() добавляет Content-Type и Bearer-токен;служебную информацию для запроса.
- apiRequest() вызывает TarelkaApi.request и нормализует ошибки;
- syncCollections() загружает все пользовательские коллекции по userId;
- Например:счета транзакции бюджеты

- initializeProtectedPage() проверяет авторизацию перед открытием защищённой страницы;
- postEntity(), patchEntity(), deleteEntity() реализуют единый CRUD для сущностей.

localStorage в ЛР2 больше не является основным хранилищем приложения, а служит локальным кэшем и средством хранения сессии. центральный результат лабы пауууу

4.3. Примеры бизнес-методов

Через TarelkaStorage реализованы операции добавления счёта, транзакции, перевода между счетами, изменения бюджета, подключения интеграции, создания правил импорта и загрузки CSV. Каждый метод создаёт новую запись в JSON Server, либо обновляет существующую запись PATCH-запросом.

-	**HTTP-операция**	-
addAccount()	POST /accounts	Создание нового счёта пользователя.
addTransaction()	POST /transactions	Сохранение дохода или расхода.
addTransfer()	POST /transactions x2	Создание связанной пары transfer_out и transfer_in.
updateTransaction()	PATCH /transactions/{id}	Редактирование транзакции.
deleteTransaction()	DELETE /transactions/{id}	Удаление записи или обеих частей перевода.
updateBudget()	PATCH /budgets/{id}	Изменение лимита категории.

toggleIntegration()	PATCH /integrations/{id}	Подключение и отключение внешнего сервиса.
importTransactions()	POST /transactions + POST /imports	Импорт операций и запись истории загрузки.

5. Реализация пользовательских страниц

5.1. Страница авторизации

Страница index.html используется для входа в систему, а register.html - для регистрации. Скрипт auth.js содержит отображение ошибок под полями, переключение видимости пароля и индикатор силы пароля. После входа пользователь перенаправляется на dashboard.html. Если пользователь уже авторизован, повторное открытие страниц входа и регистрации сразу переводит его в личный кабинет

5.2. Личный кабинет

Страница dashboard.html Во-первых, вызывает initializeProtectedPage и загружает все пользовательские данные из mock API.Во-вторых, выводит агрегированные метрики: общий баланс, доходы за месяц, расходы за месяц и процент выполнения бюджетов. В-третьих, позволяет добавлять транзакции, счета, переводы и редактировать бюджеты.

На этой же странице происходит работа с внешним API. Сначала показывается заглушка, затем вызывается loadExchangeRates, которая через TarelkaStorage.fetchExchangeRates обращается к Frankfurter API. Полученные курсы сохраняются в кэше localStorage и отображаются в виде карточек. Если сеть недоступна, приложение пытается использовать ранее сохранённые данные.

5.3. Раздел транзакций

Страница transactions.html реализует просмотр операций в виде таблицы и карточек. В файле transactions.js предусмотрены фильтры по тексту, категории, счёту, типу, статусу, сумме и диапазону дат, есть сортировка, переключение режима отображения, пагинация, просмотр подробностей, редактирование и удаление записей.

все действия на этой странице изменяют реальные данные на mock API, а не только интерфейс. После редактирования вызывается PATCH-запрос, после удаления - DELETE-запрос, а затем список строится заново на основе актуального состояния.

5.4. Отчёты

В reports.js данные для графиков формируются функцией getReportData из storage.js. На основе транзакций за выбранный период рассчитываются общие расходы и доходы, средний чек, топ-категория, тренды и прогресс бюджетов.

Графики строятся уже на клиенте, но источником служат данные, загруженные с mock API. сами графики рисуются в браузере, но данные для них приходят с учебного сервера.

5.5. Интеграции и импорт

В integrations.js объединены 3 сценария: управление подключёнными сервисами, создание правил импорта и загрузка транзакций из CSV.

Для парсинга CSV используется простая функция parseCsv, разбивающая входной текст на строки и столбцы. После чтения файла строки передаются в TarelkaStorage.importTransactions, где к ним применяются правила пользователя и выполняется пакетная запись в коллекцию transactions.

Также в приложении есть тестовый сценарий useSampleImportBtn. Он берёт заранее подготовленный CSV из data.js и позволяет быстро показать на защите, что импорт действительно работает и попадает в mock API.

6. Основные алгоритмы работы приложения

6.1. Алгоритм регистрации нового пользователя

1. Пользователь заполняет форму регистрации.
2. auth.js проверяет поля и формирует payload.
3. storage.js отправляет POST-запрос на /register.
4. Сервер возвращает accessToken и созданного пользователя.
5. Токен сохраняется в localStorage.
6. Вызывается seedUserData(), которая создаёт стартовые счета, бюджеты, транзакции, интеграции и правила импорта.
7. После этого выполняется syncCollections(), и пользователь попадает в заполненный личный кабинет.

6.2. Алгоритм входа

1. Пользователь вводит email и пароль.

2. Форма вызывает TarelkaStorage.login(email, password).
3. Происходит POST-запрос на /login.
4. Ответ содержит accessToken и профиль пользователя.
5. Метод syncCollections() получает все закрытые коллекции по userId.
6. Интерфейс открывает dashboard.html.

6.3. Алгоритм получения курсов валют

1. Личный кабинет определяет базовую валюту пользователя.
2. Метод fetchExchangeRates() формирует набор целевых валют и URL параметров.
3. Через fetch выполняется обращение к Frankfurter API.
4. Результат сохраняется в localStorage как кэш exchangeRates.
5. Карточки на dashboard.html выводят полученные значения.
6. Если запрос к внешнему API не удался, приложение использует кэш, если он есть.

6.4. Алгоритм импорта CSV

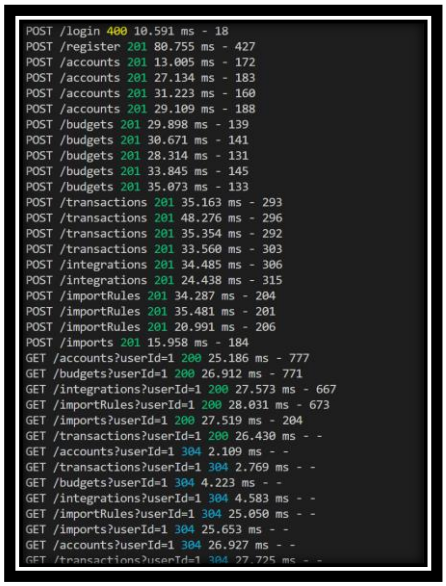
1. Пользователь выбирает CSV-файл и счёт, в который нужно загрузить операции.
2. Файл считывается методом file.text().
3. parseCsv() преобразует текст файла в массив объектов.
4. importTransactions() отбрасывает неполные строки и применяет правила из importRules.
5. Каждая подготовленная запись отправляется в transactions через addTransaction().
6. После завершения создаётся запись в imports с количеством импортированных строк и именем файла.

7. Итоги

- ✓ Требование задания: привязать ЛР1 к внешнему API средствами fetch. ==: api.js и storage.js используют fetch для mock API и Frankfurter API.
- ✓ реализовать mock API средствами JSON Server==: package.json, db.json, запуск json-server на порту 3000.

- ✓ подключить авторизацию ==: /register, /login, accessToken, Bearer, routes.json с кодом 600.
- ✓ перенести операции CRUD в API. ==: add/update/delete для счетов, транзакций, бюджетов, интеграций и импорта.
- ✓ продемонстрировать работу приложения с реальными данными. ==: курсы валют загружаются из открытого внешнего API.

Во время практической проверки проект запускался командой npm start, после чего в терминале были видны корректные логи вида POST /register 201, POST /accounts 201, POST /transactions 201 и GET /... 200. подтверждают, что создание и чтение сущностей выполняется на стороне mock API. (Коды 304, которые видны при повторных GET-запросах, не являются ошибкой и означают использование кэша браузера.)



```
POST /login 400 10.591 ms - 18
POST /register 201 80.755 ms - 427
POST /accounts 201 13.005 ms - 172
POST /accounts 201 27.134 ms - 183
POST /accounts 201 31.223 ms - 160
POST /accounts 201 29.109 ms - 188
POST /budgets 201 29.898 ms - 139
POST /budgets 201 30.671 ms - 141
POST /budgets 201 28.314 ms - 131
POST /budgets 201 33.845 ms - 145
POST /budgets 201 35.073 ms - 133
POST /transactions 201 35.163 ms - 293
POST /transactions 201 48.276 ms - 296
POST /transactions 201 35.354 ms - 292
POST /transactions 201 33.560 ms - 303
POST /integrations 201 34.485 ms - 306
POST /integrations 201 24.438 ms - 315
POST /importRules 201 34.287 ms - 204
POST /importRules 201 35.481 ms - 201
POST /importRules 201 20.991 ms - 206
POST /imports 201 15.958 ms - 184
GET /accounts?userId=1 200 25.186 ms - 777
GET /budgets?userId=1 200 26.912 ms - 771
GET /integrations?userId=1 200 27.573 ms - 667
GET /importRules?userId=1 200 28.031 ms - 673
GET /imports?userId=1 200 27.519 ms - 204
GET /transactions?userId=1 200 26.430 ms - -
GET /accounts?userId=1 304 2.109 ms - -
GET /transactions?userId=1 304 2.769 ms - -
GET /budgets?userId=1 304 4.223 ms - -
GET /integrations?userId=1 304 4.583 ms - -
GET /importRules?userId=1 304 25.050 ms - -
GET /imports?userId=1 304 25.653 ms - -
GET /accounts?userId=1 304 26.927 ms - -
GET /transactions?userId=1 304 27.725 ms - -
```

Рис1-npm start

Приложение А. Краткие фрагменты кода

Эти фрагменты показывают: единый fetch-слой, передачу токена авторизации и централизованную синхронизацию пользовательских коллекций.

```
async function request(path, options = {}) {
  const response = await fetch(`${BASE_URL}${path}`,
options);
  const payload = await parseResponse(response);
  if (!response.ok) throw new Error(extractError(payload));
  return payload;
}
function authHeaders(isJson = true) {
  const headers = {};
  if (isJson) headers['Content-Type'] = 'application/json';
```

```
  const token = getAuthToken();
  if (token) headers.Authorization = `Bearer ${token}`;
  return headers;
}
```

```
async function syncCollections(userId =
getCurrentUser()?.id) {
  const entries = await
Promise.all(COLLECTION_KEYS.map(async (key) => {
    const result = await
apiRequest(`${ENDPOINTS[key]}?userId=${encodeURIComponent(u
serId)}`, {
      headers: authHeaders(false)
    });
    return [key, Array.isArray(result) ? result : []];
  }));
}
```

8. Заключение

ЛР1 переведено на взаимодействие с mock API, развёрнутым на JSON Server. Для защищённых коллекций подключена авторизация через json-server-auth. Все основные сущности обрабатываются сетевыми запросами: выполняются операции создания, чтения, обновления и удаления данных. Дополнительно в проект встроено открытое внешнее API курсов валют.Перестроила короче архитектуру уже существующего приложения под модель клиент - API

9. Рефлексия

Прикольно что storage.js уменьшает дублирование запросов; еще впервые делала mock API и авторизацию; круто звучит тот факт,что внешнее API интегрировано в предметную область приложения, а не добавлено формально; а слово кэш который для валют звучит вообще балдежно. уже не так вайбово,что парсер корректно работает для файлов без вложенных

запятых; В целом я рада,что это закончилось, так как не думала,что настолько долго будет